PyGAD

```
# Install PyGAD (run once in Colab)
!pip install pygad

# Import required libraries
import pygad
import numpy as np

# Problem data (fixed array)
data = np.array([4, -2, 3.5, 5, -11, -4.7])

# Fitness function
def fitness_function(ga_instance, solution, solution_idx):
    fitness = np.dot(solution, data)
    return fitness

# Create GA instance
ga_instance = pygad.GA(
    num_generations=100,
    num_parents_mating=4,
    fitness_func=fitness_function,
    sol_per_pop=10,
    num_genes=len(data),
    init_range_low=-5,
    init_range_high=5,
    mutation_percent_genes=20
)

# Run the Genetic Algorithm
ga_instance.run()

# Get best solution
solution, solution_fitness, solution_idx = ga_instance.best_solution()

# Print results
print("Best solution (weights):")
print(solution)

print("\nBest fitness value:")
print(solution_fitness)

print("\nFound at generation:")
print(ga_instance.best_solution_generation)

# Plot fitness curve
ga_instance.plot_fitness()
```

```
Collecting pygad
  Downloading pygad-3.5.0-py3-none-any.whl.metadata (20 kB)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dis
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.1
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.1
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/pytho
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-pa
Downloading pygad-3.5.0-py3-none-any.whl (89 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 89.6/89.6 kB 5.2 MB/s eta 0:00:0
Installing collected packages: pygad
Successfully installed pygad-3.5.0
Best solution (weights):
[ 13.61231977  -3.26762531   7.5014391   17.52316453 -25.82274678
 -17.17243815]

Best fitness value:
539.6160630377353

Found at generation:
100
/usr/local/lib/python3.12/dist-packages/pygad/visualize/plot.py:120: UserWar
```

DEAP

```
# Install DEAP (run once in Google Colab)
!pip install deap

import random
import numpy as np
from deap import base, creator, tools, algorithms

# Fixed data array
data = np.array([4, -2, 3.5, 5, -11, -4.7])

# Create Fitness and Individual classes
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)

# Toolbox setup
toolbox = base.Toolbox()

# Gene: random float between -5 and 5
toolbox.register("attr_float", random.uniform, -5, 5)

# Individual: list of genes
toolbox.register("individual", tools.initRepeat, creator.Individual,
                 toolbox.attr_float, n=len(data))

# Population: list of individuals
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

# Fitness function
```

```python
def evaluate(individual):
    return (np.dot(individual, data),)

toolbox.register("evaluate", evaluate)

# Genetic operators
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.2)
toolbox.register("select", tools.selTournament, tournsize=3)

# Create population
population = toolbox.population(n=10)

# Run Genetic Algorithm
population, logbook = algorithms.eaSimple(
    population,
    toolbox,
    cxpb=0.7,     # Crossover probability
    mutpb=0.2,    # Mutation probability
    ngen=50,      # Number of generations
    verbose=False
)

# Get best solution
best_individual = tools.selBest(population, k=1)[0]

# Print results
print("Best solution (weights):")
print(best_individual)

print("\nBest fitness value:")
print(best_individual.fitness.values[0])
```

```
Requirement already satisfied: deap in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packag
Best solution (weights):
[7.256741113297936, -3.636959011815059, 8.982811526834695, 8.654158768751733,

Best fitness value:
229.89803478116474
/usr/local/lib/python3.12/dist-packages/deap/creator.py:185: RuntimeWarning:
  warnings.warn("A class named '{0}' has already been created and it "
/usr/local/lib/python3.12/dist-packages/deap/creator.py:185: RuntimeWarning:
  warnings.warn("A class named '{0}' has already been created and it "
```

Genetical Algorithm Numerical Example

1. define fitness function

```
def fitness(x):
    return 12*x - x*x
```

2. Binary to Decimal

```
def binary_to_decimal(binary):
    return int(binary, 2)

def decimal_to_binary(x):
    return format(x, '03b')
```

3. Initial Population

```
population = ["0011", "0101", "0110", "1001", "1010"]
```

4. Fitness evaluation

```
fitness_values = {}

for chrom in population:
    x = binary_to_decimal(chrom)
    fitness_values[chrom] = fitness(x)

print("Fitness values:", fitness_values)
```

```
Fitness values: {'0011': 27, '0101': 35, '0110': 36, '1001': 27, '1010': 20}
```

5. Selection (Top 2)

```
parents = sorted(fitness_values, key=fitness_values.get, reverse=True)[:2]
p1, p2 = parents
print("Selected Parents:", p1, p2)
```

```
Selected Parents: 0110 0101
```

6. Single-point crossover

```
crossover_point = 1

child1 = p1[:crossover_point] + p2[crossover_point:]
child2 = p2[:crossover_point] + p1[crossover_point:]

print("After crossover:", child1, child2)
```

```
After crossover: 0101 0110
```

7. Mutation

```
def mutate(chromosome, bit=2):
    mutated = list(chromosome)
    mutated[bit] = '1' if mutated[bit] == '0' else '0'
    return ''.join(mutated)

child1 = mutate(child1)
child2 = mutate(child2)

print("After mutation:", child1, child2)
```

```
After mutation: 0111 0100
```

8. Final evaluation

```
final_population = [child1, child2]

best_x = -1
best_value = -1

for chrom in final_population:
    x = binary_to_decimal(chrom)
    value = fitness(x)
    if value > best_value:
        best_value = value
        best_x = x

print("Best solution:")
print("x =", best_x)
print("f(x) =", best_value)
```

```
Best solution:
x = 7
f(x) = 35
```

Particle Swarm Optimization (PSO)

1. Fitness function

```
def fitness(x):
    return (x - 3)**2
```

2. Initialize particles

```
particles = [
    {"x": 0.5, "v": 0.0},
    {"x": 5.5, "v": 0.0},
    {"x": 2.0, "v": 0.0}
]

pbest = [
    {"x": p["x"], "v": p["v"]} for p in particles
]
```

3. PSO parameters

```
w = 0.5
c1 = 1.5
c2 = 1.5
r1 = 0.5
r2 = 0.5

max_iterations = 30
tolerance = 1e-4
```

4. PSO main loop

```
for iteration in range(max_iterations):

    # Find global best
    gbest = min(particles, key=lambda p: fitness(p["x"]))
    gbest_x = gbest["x"]

    # Stop if sufficiently close
    if fitness(gbest_x) < tolerance:
```

```
            break

        # Update velocity and position
        for i in range(len(particles)):
            x = particles[i]["x"]
            v = particles[i]["v"]
            pb = pbest[i]["x"]

            v_new = (
                w * v
                + c1 * r1 * (pb - x)
                + c2 * r2 * (gbest_x - x)
            )

            x_new = x + v_new

            # Keep within bounds
            x_new = max(0, min(6, x_new))

            particles[i]["v"] = v_new
            particles[i]["x"] = x_new

            # Update personal best
            if fitness(x_new) < fitness(pbest[i]["x"]):
                pbest[i]["x"] = x_new

        print(
            f"Iteration {iteration+1}: "
            f"gBest = {round(gbest_x, 4)}, "
            f"f(x) = {round(fitness(gbest_x), 6)}"
        )
```

```
Iteration 1: gBest = 2.0, f(x) = 1.0
Iteration 2: gBest = 2.875, f(x) = 0.015625
Iteration 3: gBest = 3.125, f(x) = 0.015625
Iteration 4: gBest = 3.0625, f(x) = 0.003906
Iteration 5: gBest = 3.0781, f(x) = 0.006104
Iteration 6: gBest = 3.0742, f(x) = 0.005508
Iteration 7: gBest = 2.9272, f(x) = 0.005293
Iteration 8: gBest = 2.9552, f(x) = 0.002007
Iteration 9: gBest = 2.9852, f(x) = 0.000219
Iteration 10: gBest = 2.9777, f(x) = 0.000497
Iteration 11: gBest = 2.9796, f(x) = 0.000417
```

5. Final output

```
print("\nFinal Answer")
print("Optimal x =", round(gbest_x, 6))
print("Minimum f(x) =", round(fitness(gbest_x), 6))
```

```
Final Answer
```

```
Optimal x = 2.994074
Minimum f(x) = 3.5e-05
```

PSO Using PyGAD

```
import pygad
import numpy as np

def fitness_func(ga_instance, solution, solution_idx):
    x = solution[0]
    return -((x - 3) ** 2)   # negative for minimization

def on_generation(ga_instance):
    generation = ga_instance.generations_completed
    solution, solution_fitness, solution_idx = ga_instance.best_solution()

    print(
        f"Generation {generation}: "
        f"Best x = {round(solution[0], 4)}, "
        f"f(x) = {round(-solution_fitness, 6)}"
    )


ga_instance = pygad.GA(
    num_generations=30,
    num_parents_mating=3,
    fitness_func=fitness_func,

    sol_per_pop=3,
    num_genes=1,
    init_range_low=0,
    init_range_high=6,

    parent_selection_type="tournament",
    K_tournament=2,

    crossover_type="single_point",

    mutation_type="random",
    mutation_percent_genes=100,
    mutation_by_replacement=True,
    random_mutation_min_val=-0.5,
    random_mutation_max_val=0.5,

    on_generation=on_generation
)

ga_instance.run()


solution, solution_fitness, solution_idx = ga_instance.best_solution()
```

```python
print("\nFinal Answer")
print("Optimal x =", round(solution[0], 6))
print("Minimum f(x) =", round(-solution_fitness, 6))
```

```
Generation 1: Best x = 2.775, f(x) = 0.050604
Generation 2: Best x = 2.775, f(x) = 0.050604
Generation 3: Best x = 2.775, f(x) = 0.050604
Generation 4: Best x = 2.775, f(x) = 0.050604
Generation 5: Best x = 2.775, f(x) = 0.050604
Generation 6: Best x = 2.775, f(x) = 0.050604
Generation 7: Best x = 2.775, f(x) = 0.050604
Generation 8: Best x = 2.775, f(x) = 0.050604
Generation 9: Best x = 2.775, f(x) = 0.050604
Generation 10: Best x = 2.775, f(x) = 0.050604
Generation 11: Best x = 2.775, f(x) = 0.050604
Generation 12: Best x = 2.775, f(x) = 0.050604
Generation 13: Best x = 2.775, f(x) = 0.050604
Generation 14: Best x = 2.775, f(x) = 0.050604
Generation 15: Best x = 2.775, f(x) = 0.050604
Generation 16: Best x = 2.775, f(x) = 0.050604
Generation 17: Best x = 2.775, f(x) = 0.050604
Generation 18: Best x = 2.775, f(x) = 0.050604
Generation 19: Best x = 2.775, f(x) = 0.050604
Generation 20: Best x = 2.775, f(x) = 0.050604
Generation 21: Best x = 2.775, f(x) = 0.050604
Generation 22: Best x = 2.775, f(x) = 0.050604
Generation 23: Best x = 2.775, f(x) = 0.050604
Generation 24: Best x = 2.775, f(x) = 0.050604
Generation 25: Best x = 2.775, f(x) = 0.050604
Generation 26: Best x = 2.775, f(x) = 0.050604
Generation 27: Best x = 2.775, f(x) = 0.050604
Generation 28: Best x = 2.775, f(x) = 0.050604
Generation 29: Best x = 2.775, f(x) = 0.050604
Generation 30: Best x = 2.775, f(x) = 0.050604

Final Answer
Optimal x = 2.775046
Minimum f(x) = 0.050604
```